

HOW_TO_REPRODUCE

Megang Nkamga Junile Staures

This document lists the exact steps to reproduce `submission.csv` and every figure/table referenced in `REPORT.pdf`, starting from a clean clone of the repository.

1 Environment

```
python3 -m venv ml.venv
source ml.venv/bin/activate          # macOS/Linux
pip install -r requirements.txt
```

Key packages: `pandas`, `numpy`, `scikit-learn`, `xgboost`, `lightgbm`, `catboost`, `imbalanced-learn`, `shap`, `matplotlib`, `seaborn`, `scipy`.

2 Data placement

Place the two provided files under `data/`:

```
data/d2assignment_dataset.csv.gz
data/d2assignment_test.csv.gz
```

3 Random seed

`random_state = 42` is fixed everywhere randomness occurs: the train/validation split, every k-fold object, SMOTE, every model constructor, and `RandomizedSearchCV`. Re-running the scripts below reproduces all reported numbers exactly.

4 Step 1: EDA and preprocessing

```
jupyter notebook eda_preprocessing_pipeline.ipynb
```

Reads the two raw `.csv.gz` files and produces:

- `data/train_clean.csv`, `data/test_clean.csv` – cleaned, feature-engineered, one-hot encoded ($96,112 \times 984$ and $23,888 \times 983$ respectively, including `row_id` and, for train, `target`);

- `Figures/target_distribution.pdf`;
- `Figures/correlation_heatmap_raw_data.pdf`, `Figures/correlation_heatmap_clean_data.pdf`;
- `Figures/visualize_offenders.pdf` (KS drift plots).

This step also prints the leakage-column list, the `app_kind` constant-column check, and the missingness/default-rate table used in Section 2 of `REPORT.pdf`.

5 Step 2: Modeling

jupyter notebook `loan_default_pipeline.ipynb`

Reads `data/train_clean.csv` / `data/test_clean.csv` and produces:

- majority-class baseline (accuracy and ROC-AUC);
- 5-fold and 10-fold CV comparison across seven models (Logistic Regression, Random Forest, LightGBM, XGBoost, XGBoost Weighted, CatBoost, and CatBoost Balanced);
- SMOTE and class-weighting follow-up experiments (10-fold CV);
- tuned CatBoost via `RandomizedSearchCV` (30 iterations, 10-fold CV);
- `Figures/shap_summary_plot.pdf`, `Figures/shap_feature_importance_bar.pdf`;
- `Figures/threshold_metrics.pdf`, `Figures/confusion_matrix.pdf`;
- `val_submission.csv`, `validation_labels.csv` (held-out predictions and true labels, for grading verification);
- `submission.csv` (final predictions on the 23,888-row test set, from the model refit on train + validation combined).

Expected runtime: well over an hour on a standard laptop CPU. This step uses 10-fold CV throughout (model comparison, SMOTE evaluation, class-weighting evaluation, and the 30-iteration hyperparameter search) rather than switching to 5-fold for speed, so every reported number in `REPORT.pdf` is directly reproducible at the stated fold count. Rough breakdown observed on this machine:

- data load + train/val split + majority baseline + 5-fold and 10-fold model comparison (7 models): ≈ 20 minutes;
- SMOTE experiment, 10-fold: ≈ 20 minutes;
- CatBoost/XGBoost class-weighting comparison, 10-fold: a few minutes;
- hyperparameter search (30 iterations, 10-fold): the single largest cost.

If reproducing on slower hardware, expect this step to take considerably longer; there is no shortcut built into the script for a faster run other than manually swapping `skf_10` for `skf_5` and reducing `n_iter`, which would no longer reproduce the exact numbers in `REPORT.pdf`.

6 Step 3: Verify the submission

```
python score_submission.py \  
    --submission val_submission.csv \  
    --labels validation_labels.csv
```

This should print a ROC-AUC matching the "Final validation" AUC reported in `REPORT.pdf` (0.6953). If it doesn't match, Step 2 was likely run with a different `random_state` or a stale `data/train_clean.csv/data/test_clean.csv`; re-run Step 1 first.

7 Output checklist

After both steps, the following should exist and match `REPORT.pdf`:

- `data/train_clean.csv`, `data/test_clean.csv`
- Console output: majority-class baseline (accuracy 0.8565, ROC-AUC 0.5)
- `Figures/*.pdf` (7 files, listed above)
- `val_submission.csv`, `validation_labels.csv`
- `submission.csv`: 23,888 rows, columns `row_id`, `predicted_probability`, no duplicate `row_id` values, probabilities in $[0, 1]$